4 MARKS

1. Define Data Visualization. Mention its importance in data analysis.

Data Visualization is the process of representing data in graphical or pictorial formats such as charts, graphs, and plots. It helps convert complex numerical data into visual insights.

Importance:

1. Identifies Patterns and Trends: Helps quickly observe trends, correlations, and outliers.
2. Improves Understanding: Makes complex datasets easier to interpret.
3. Supports Decision Making: Clear visuals help in making data-driven decisions.
4. Better Communication: Visuals present results more effectively to users who may not be familiar with raw data.

2. What is Matplotlib? Write its basic syntax for creating a simple line plot.

Matplotlib is a popular Python library used for creating static, animated, and interactive visualizations. It provides low-level control for generating various plots such as line, bar, scatter, and histogram.

Basic Syntax for a Simple Line Plot:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
plt.plot(x, y)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Simple Line Plot")
plt.show()
```

3. What are the advantages of using Seaborn over Matplotlib?

1. Better Aesthetics: Seaborn provides attractive default themes and color palettes.
2. High-Level Interface: It reduces code complexity with simpler functions for complex plots.
3. Built-in Statistical Plots: Supports plots like boxplot, violin plot, pairplot, and heatmaps easily.
4. Easy Integration with Pandas: Works smoothly with DataFrame structures, reducing manual data handling.

5. Name any four common visualization types available in Pandas.

Data Visualization is the process of representing data in graphical or pictorial formats such as charts, graphs, and plots. It helps convert complex numerical data into visual insights.

Four common visualization types:

1. Line Plot
2. Bar Plot
3. Histogram
4. Pie chart

5. Write a Python program to create and display a simple scatter plot using Matplotlib.

```
import matplotlib.pyplot as plt

x = [10, 20, 30, 40, 50]
y = [5, 15, 25, 20, 30]
plt.scatter(x, y)
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Simple Scatter Plot")
plt.show()
```

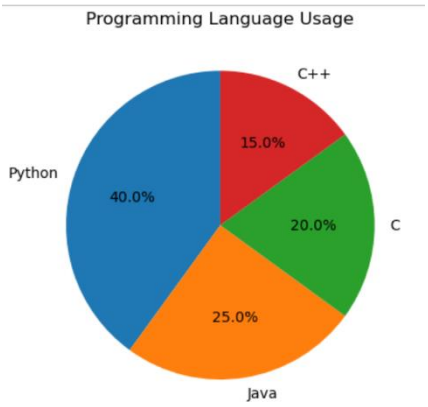
6 MARKS

1. Write a Python program to create a pie chart using Matplotlib.

A pie chart represents data proportions in circular slices.

```
import matplotlib.pyplot as plt

labels = ['Python', 'Java', 'C', 'C++']
sizes = [40, 25, 20, 15]

plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title("Programming Language Usage")
plt.show()
```

OUTPUT:



---

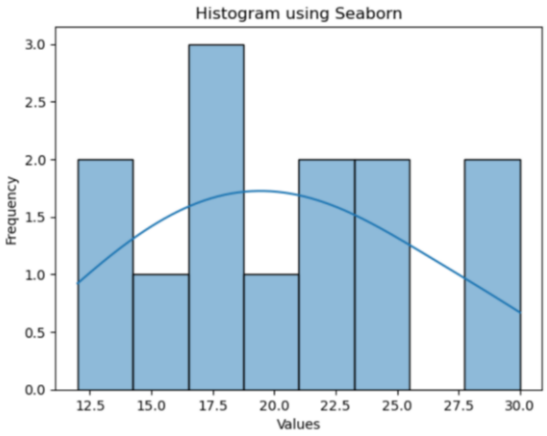
2. Write a Python program to create a histogram using Seaborn.

A histogram shows the frequency distribution of numeric data.

```
import seaborn as sns
import matplotlib.pyplot as plt

data = [12, 15, 20, 22, 18, 17, 13, 25, 30, 28, 18, 22, 24]
sns.histplot(data, bins=8, kde=True)
plt.xlabel("Values")
```

```
plt.ylabel("Frequency")
plt.title("Histogram using Seaborn")
plt.show()
```

Output:



3. Analyse how visualization customization (titles, labels, legends, colors) enhances data readability.

Customization makes visualizations easier to understand and interpret.

1. Titles
    - Provide a clear description of what the chart represents.
    - Helps viewers quickly understand the purpose of the graph.
2. X-axis and Y-axis Labels
    - Indicate what the horizontal and vertical axes measure.
    - Prevent confusion and give context, especially in numeric datasets.
3. Legends
    - Useful when multiple lines or colors appear in the same chart.
    - Help identify different categories or groups without crowding the graph.
4. Colors
    - Highlight differences between categories or data groups.
    - Make the visualization visually appealing and improve memory retention.
    - Using consistent color schemes prevents misinterpretation.

4. Write short notes on any three Pandas plot types.

(i) Line Plot

- Used to show trends over time or continuous values.
  df.plot(kind='line')

(ii) Bar Plot

- Compares categories or groups using rectangular bars.
- Useful for discrete data.

  df.plot(kind='bar')

(iii) Histogram

- Shows the distribution of numeric data.

  df.plot(kind='hist')

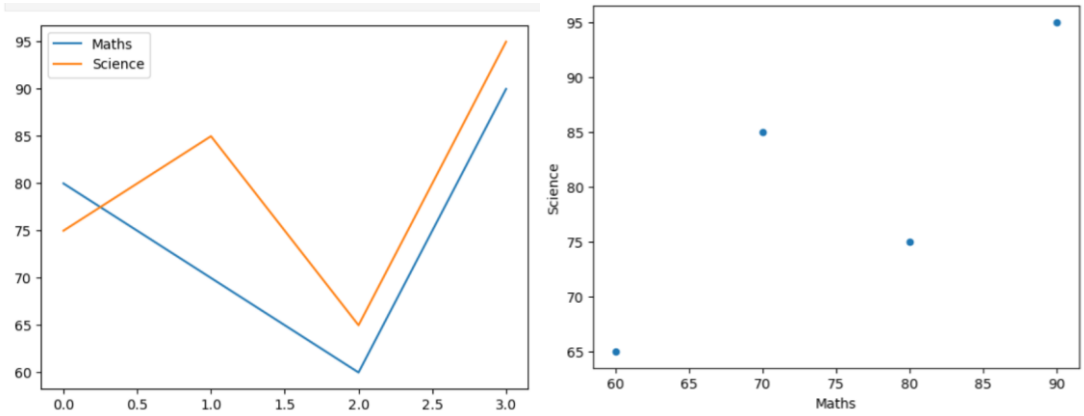
5.Write a Python program using Pandas to create a Line plot and a Scatter plot for a dataset showing students marks in Mathematics and Science.

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt


data = {
   'Maths': [80, 70, 60, 90],
   'Science': [75, 85, 65, 95]
}
df = pd.DataFrame(data)
df.plot(kind='line')
plt.show()
df.plot(kind='scatter', x='Maths', y='Science')
```

plt.show()

Output:



10 MARKS

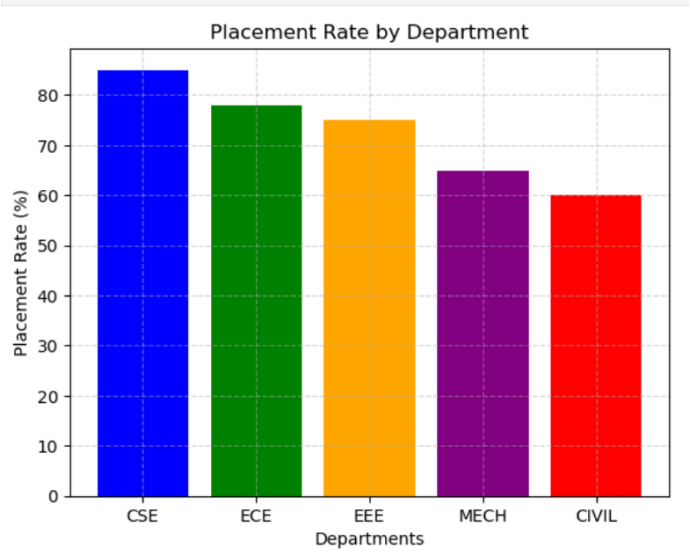
1.Using the following dataset, create a Pandas DataFrame and then plot a bar chart to show the placement rate of each department. Include a title, axis labels, grid lines, and custom bar colors.

data = {
'Department': ['CSE', 'ECE', 'EEE', 'MECH', 'CIVIL'],
'Placement_Rate': [85, 78, 75, 65, 60]
}

Answer:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Department': ['CSE', 'ECE', 'EEE', 'MECH', 'CIVIL'],
    'Placement_Rate': [85, 78, 75, 65, 60]
}
df = pd.DataFrame(data)
# Bar Chart
plt.bar(df['Department'], df['Placement_Rate'],
```

```
    color=['blue', 'green', 'orange', 'purple', 'red'])

plt.title("Placement Rate by Department")
plt.xlabel("Departments")
plt.ylabel("Placement Rate (%)")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



2.Evaluate different plot types in Pandas and provide with an example

Pandas provides several built-in plot types using the .plot() function (based on Matplotlib).
Below are some important plots:

1. Line Plot

Used for trends and time-series.

df.plot(kind='line')

Best for showing growth, variation, or continuous change.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Maths': [80, 70, 60, 90],
    'Science': [75, 85, 65, 95]
}
df = pd.DataFrame(data)
df.plot(kind='line')
plt.show()
```

2. Bar Plot

Used for comparing categories.

```
df.plot(kind='bar')
```

Helps compare values across groups.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    'Maths': [80, 70, 60, 90],
    'Science': [75, 85, 65, 95]
}
df = pd.DataFrame(data)
df.plot(kind='bar')
plt.show()
```

3. Histogram

Shows distribution of numeric data.

df['Marks'].plot(kind='hist', bins=10)

Useful for understanding spread and frequency.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
   'Marks': [45, 55, 60, 65, 70, 75, 80, 85, 90, 95]
}
df = pd.DataFrame(data)
df['Marks'].plot(kind='hist', bins=5)
plt.show()
```

4. Scatter Plot

Shows relationship between two variables.

df.plot(kind='scatter', x='Height', y='Weight')

Helps identify correlation.

Example:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
   'Height': [150, 155, 160, 165, 170, 175],
   'Weight': [50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
df.plot(kind='scatter', x='Height', y='Weight')
plt.show()
```

5. Pie Chart

Shows proportion of categories.

df['Weight'].plot(kind='pie')

Useful for percentage comparison.

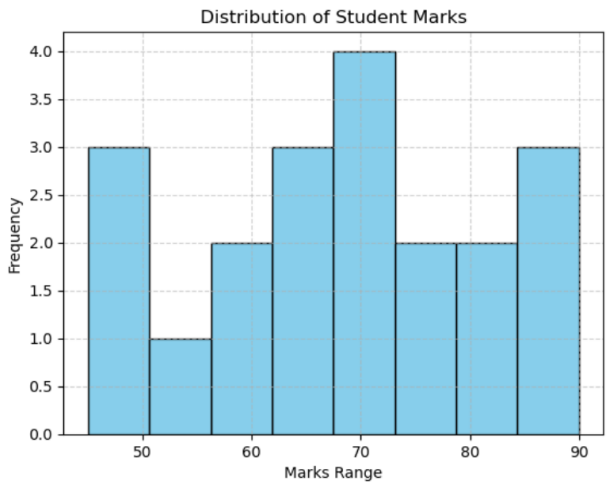
Example:

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Height': [150, 155, 160, 165, 170, 175],
   'Weight': [50, 55, 60, 65, 70, 75]
}

df = pd.DataFrame(data)

df['Weight'].plot(
   kind='pie',
   labels=df['Height'],
   autopct='%1.1f%%'
)

plt.show()
```

3.Using the following dataset of student marks, create a histogram to show the distribution of marks. Customize the histogram by adding a title, x-axis and y-axis labels, grid lines, and set appropriate number of bins and bar colors
marks = [45, 67, 78, 50, 62, 70, 80, 55, 68, 90, 72, 60, 85, 77, 49, 66, 88, 73, 59, 81]

Answer:

```
import matplotlib.pyplot as plt

marks = [45, 67, 78, 50, 62, 70, 80, 55, 68, 90, 72, 60, 85, 77, 49, 66, 88, 73, 59, 81]

plt.hist(marks, bins=8, color='skyblue', edgecolor='black')

plt.title("Distribution of Student Marks")

plt.xlabel("Marks Range")

plt.ylabel("Frequency")

plt.grid(True, linestyle='--', alpha=0.6)

plt.show()
```



Explanation:

- plt.hist() creates histogram.
- bins=8 divides data into 8 ranges.
- Colors, labels, grid enhance readability.
- Shows how marks are distributed across the class.

4.Create a comparative visualization report using Pandas plot types.
Given a dataset with columns — Department, Male_Students, Female_Students, Placement_Rate.
Perform:

1.Pie chart – Gender ratio

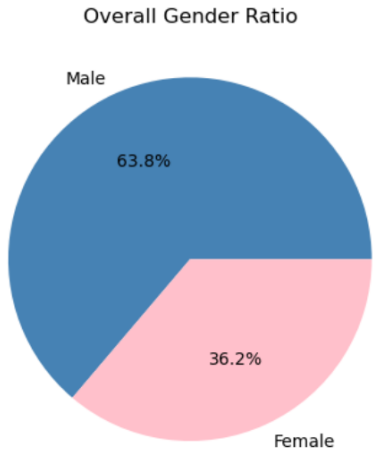
2.Line chart – Placement rate trend
3.Custom styling (color, grid, title, layout)

Answer:

Dataset Structure: Department, Male_Students, Female_Students, Placement_Rate

(i) Pie Chart – Gender Ratio

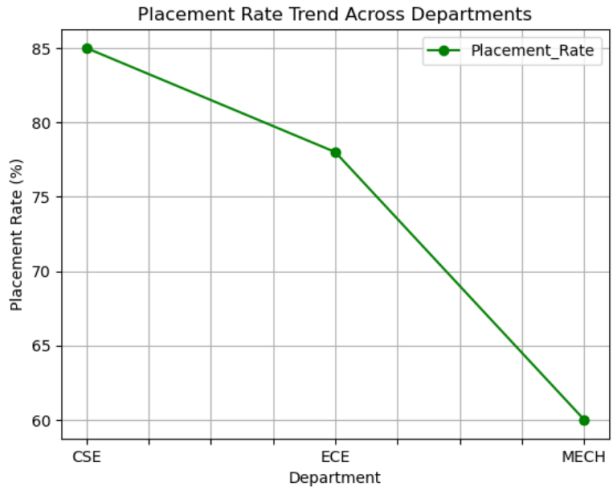
```
import pandas as pd
import matplotlib.pyplot as plt

data = {
   'Department': ['CSE', 'ECE', 'MECH'],
   'Male_Students': [120, 100, 150],
   'Female_Students': [80, 90, 40],
   'Placement_Rate': [85, 78, 60]
}
df = pd.DataFrame(data)

plt.pie([df['Male_Students'].sum(), df['Female_Students'].sum()],
        labels=['Male', 'Female'],
        autopct='%1.1f%%',
        colors=['steelblue', 'pink'])

plt.title("Overall Gender Ratio")
plt.show()
```



(ii) Line Chart – Placement Rate Trend

```
df.plot(x='Department', y='Placement_Rate',
        kind='line', marker='o', color='green')

plt.title("Placement Rate Trend Across Departments")
plt.xlabel("Department")
plt.ylabel("Placement Rate (%)")
plt.grid(True)
plt.show()
```

Output:



Styling Features Used:

- Custom colors
- Markers
- Grid lines
- Title and axis labels
- Pie chart percentage formatting

5.Explain the important methods used in Matplotlib for creating and customizing plots with examples.

Matplotlib provides many functions for building and customizing graphs.

---

1. plt.plot() – Line Plot

Creates a line graph.

plt.plot(x, y)

---

2. plt.scatter() – Scatter Plot

Shows relationship between two variables.

plt.scatter(x, y)

---

3. plt.bar() – Bar Chart

Used for categorical comparison.

plt.bar(categories, values)

---

4. plt.hist() – Histogram

Shows distribution.

plt.hist(data, bins=10)

---

5. plt.title() – Add Title

Adds title to plot.

plt.title("Sales Growth")

---

6. plt.xlabel() and plt.ylabel() – Axis Labels

```
plt.xlabel("Months")
plt.ylabel("Revenue")
```

---

7. plt.legend() – Add Legend

plt.legend(["2023 Sales", "2024 Sales"])

---

8. plt.grid() – Add Grid Lines

plt.grid(True, linestyle='--')

---

9. plt.show() – Display the Plot

plt.show()

Example:

```
import matplotlib.pyplot as plt

# Sample data
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
sales_2023 = [200, 250, 300, 280, 350]
sales_2024 = [220, 270, 320, 300, 380]

# Line Plot
plt.plot(months, sales_2023, label='2023 Sales')

# Scatter Plot
plt.scatter(months, sales_2024, label='2024 Sales')

# Bar Chart
plt.bar(months, sales_2023, alpha=0.3)

# Histogram
plt.hist(sales_2024, bins=5)

# Add title
plt.title("Sales Growth Comparison")
```

```
# Axis labels
plt.xlabel("Months")
plt.ylabel("Revenue")

# Legend
plt.legend()

# Grid
plt.grid(True, linestyle='--')

# Show plot
plt.show()
```